

TEMPLATING FILES:

- Ansible Engine has a number of modules that can be used to modify existing files.
- These include **lineinfile** and **blockinfile**, among others.
- However, they are not always easy to use effectively and correctly.
- A much more powerful way to manage files is to template them.
- With this method, you can write a template configuration file that is automatically customized for the managed host when the file is deployed, using Ansible variables and facts.
- This can be easier to control and is less error-prone.

JINJA2 Template:

- Ansible uses the Jinja2 templating system for template files.
- Ansible also uses Jinja2 syntax to reference variables in playbooks.

BUILDING A JINJA2 TEMPLATE:

- A Jinja2 template is composed of multiple elements:
 - data, variables, and expressions.
- Those variables and expressions are replaced with their values when the Jinja2 template is rendered.
- The variables used in the template can be specified in the vars section of the playbook.
- It is possible to use the managed hosts' facts as variables on a template.
- Variables and logic expressions are placed between tags, or delimiters.
- Jinja2 templates use **{% EXPR %}** for expressions or logic.
- while **{{ EXPR }}** are used for outputting the results of an expression or a variable to the end user.
- Use **{# COMMENT #}** syntax to enclose comments that should not appear in the final file.

`{# /etc/hosts line #}` --> is for comments

`{{ ansible_facts['default_ipv4']['address'] }}` --> used to call facts values.

NOTE THAT

- A file containing a Jinja2 template does not need to have any specific file extension (for example, .j2).
- However, providing such a file extension may make it easier for you to remember that it is a template file.

DEPLOYING JINJA2 TEMPLATES:

- Jinja2 templates are a powerful tool to customize configuration files to be deployed on the managed hosts.
- When the Jinja2 template for a configuration file has been created, it can be deployed to the managed hosts using the template module, which supports the transfer of a local file on the control node to the managed hosts.
- To use the template module, use the following syntax.
 - The value associated with the src key specifies the source Jinja2 template.
 - The value associated with the dest key specifies the file to be created on the destination hosts

```
tasks:
```

```
- name: template render
```

```
  template:
```

```
    src: /tmp/j2-template.j2
```

```
    dest: /tmp/dest-config-file.txt
```

- The template module also allows you to specify the owner (the user that owns the file), group, permissions, and SELinux context of the deployed file, just like the file module.
- It can also take a validate option to run an arbitrary command (such as visudo -c) to check the syntax of a file for correctness before copying it into place.

MANAGING TEMPLATED FILES:

- To avoid having system administrators modify files deployed by Ansible, it is a good practice to include a comment at the top of the template to indicate that the file should not be manually edited.
- One way to do this is to use the Ansible managed string set in the `ansible_managed` directive. This is not a normal variable but can be used as one in a template.
- The `ansible_managed` directive is set in the `ansible.cfg` file:

`ansible_managed = Ansible managed`

- To include the `ansible_managed` string inside a Jinja2 template, use the following syntax:

```
{{ ansible_managed }}
```

CONTROL STRUCTURES:

- You can use Jinja2 control structures in template files to reduce repetitive typing, to enter entries for each host in a play dynamically, or conditionally insert text into a file

Using Loops

- Jinja2 uses the `for` statement to provide looping functionality.

EXAMPLE

```
``yaml
- name: /etc/hosts is up to date

hosts: all

gather_facts: yes

tasks:

- name: Deploy /etc/hosts

  template:

    src: templates/hosts.j2

    dest: /etc/hosts
````
```

## hosts.j2

```bash

```
{% for host in groups['all'] %}

{{ hostvars['host']['ansible_facts']['default_ipv4']['address'] }} {{
hostvars['host']['ansible_facts']['fqdn'] }} {{ hostvars['host']['ansible_facts']['hostname'] }}

{% endfor %}

```
```

*IMPORTANT: You can use Jinja2 loops and conditionals in Ansible templates, but not in Ansible Playbooks*

## Using Conditionals:

- Jinja2 uses the if statement to provide conditional control.
- This allows you to put a line in a deployed file if certain conditions are met.

```
```bash

    {% if ansible_os_family == "RedHat" %}

    yum install httpd

    {% else %}

    apt-get install apache2

    {% endif %}

```
```

## VARIABLE FILTERS

- Jinja2 provides filters which change the output format for template expressions (for example, to JSON).
- There are filters available for languages such as **YAML and JSON**.
- The **to\_json** filter formats the expression output using JSON.
- The **to\_yaml** filter formats the expression output using YAML

```
{{ output | to_json }}
```

```
{{ output | to_yaml }}
```

- Additional filters are available, such as the `to_nice_json` and `to_nice_yaml`.
- These filters, which format the expression output in either JSON or YAML human readable format

```
{{ output | to_nice_json }}
```

```
{{ output | to_nice_yaml }}
```

- Both the `from_json` and `from_yaml` filters expect strings in either JSON or YAML format, respectively, to parse them.

```
{{ output | from_json }}
```

```
{{ out put | from_yaml }}
```

## Filters\_EXAMPLE

```
``yaml
```

```
- name: Show variable in different formats
```

```
hosts: localhost
```

```
gather_facts: no
```

```
vars:
```

```
mydata:
```

```
name: Muhammad
```

```
role: Engineer
```

```
skills: ["Ansible", "Kubernetes", "AWS"]
```

```
tasks:
```

```
- name: Show as JSON
```

```
debug:
```

```
msg: "{{ mydata | to_json }}"
```

```
- name: Show as YAML
```

```
debug:
```

```
msg: "{{ mydata | to_yaml }}"
```

- name: Show as pretty JSON

debug:

msg: "{{ mydata | to\_nice\_json }}"

- name: Show as pretty YAML

debug:

msg: "{{ mydata | to\_nice\_yaml }}"

...

### JINJA2 Template Example

- Jinja2 template:

```bash

Welcome to: {{ ansible_facts['fqdn'] }}

System total memory: {{ ansible_facts['memtotal_mb'] }} MiB.

System processor count: {{ ansible_facts['processor_count'] }}

...

- **playbook:**

```yaml

- name: Configure system

hosts: rhelnode

tasks:

- name: Configure a custom /etc/motd

template:

src: motd.j2

dest: /etc/motd

owner: root

group: root

mode: 0644

- name: Check file exists

stat:

path: /etc/motd

register: motd

- name: Display stat results

debug:

var: motd

- name: Copy custom /etc/issue file

copy:

src: files/issue

dest: /etc/issue

owner: root

group: root

mode: 0644

- name: Ensure /etc/issue.net is a symlink to /etc/issue

file:

src: /etc/issue

dest: /etc/issue.net

state: link

owner: root

group: root

force: yes

...

**/etc/issue :** Displays a message before the login prompt on local terminals (TTYs).

#When you log in via console (e.g., Ctrl+Alt+F2), the contents of **/etc/issue** are shown right before the login: prompt.

## SUMMARY

- You can use Jinja2 templates to dynamically construct files for deployment.
- A Jinja2 template is usually composed of two elements: variables and expressions. Those variables and expressions are replaced with values when the Jinja2 template is rendered.
- Jinja2 filters transform template expressions from one kind or format of data into another.